

QKD

September 17, 2026

[]: Week-02 : Quantum Key Distribution : (Day 08) -RAJIV KUMAR YADAV

```
[1]: import qiskit
import random

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

print("Qiskit version:", qiskit.__version__)

# =====
# BB84 QUANTUM KEY DISTRIBUTION
# =====

N = 20                      # Number of transmitted qubits
EVE_PRESENT = False        # Change to True to simulate Eve

simulator = AerSimulator()

# =====
# 1. ALICE GENERATES RANDOM BITS
# =====

alice_bits = [
    random.randint(0, 1)
    for _ in range(N)
]

# Alice randomly chooses:
# 0 = Z basis
# 1 = X basis

alice_bases = [
    random.randint(0, 1)
```

```

        for _ in range(N)
    ]

    print("\nAlice's bits:")
    print(alice_bits)

    print("\nAlice's bases:")
    print(alice_bases)

# =====
# 2. ALICE PREPARES QUBITS
# =====

circuits = []

for bit, basis in zip(
    alice_bits,
    alice_bases
):

    qc = QuantumCircuit(1, 1)

    # Encode bit
    if bit == 1:
        qc.x(0)

    # Change to X basis
    if basis == 1:
        qc.h(0)

    circuits.append(qc)

# =====
# 3. EVE INTERCEPTS QUBITS (OPTIONAL)
# =====

if EVE_PRESENT:

    eve_bases = [
        random.randint(0, 1)
        for _ in range(N)
    ]

    print("\nEve's bases:")

```

```

print(eve_bases)

# Eve measures every qubit
# in her randomly selected basis

for i in range(N):

    if eve_bases[i] == 1:
        circuits[i].h(0)

    circuits[i].measure(0, 0)

    result = simulator.run(
        transpile(circuits[i], simulator),
        shots=1
    ).result()

    measured = list(
        result.get_counts().keys()
    )[0]

    eve_bit = int(measured)

    # Eve prepares a new qubit
    new_qc = QuantumCircuit(1, 1)

    if eve_bit == 1:
        new_qc.x(0)

    if eve_bases[i] == 1:
        new_qc.h(0)

    circuits[i] = new_qc

# =====
# 4. BOB RANDOMLY CHOOSES MEASUREMENT BASES
# =====

bob_bases = [
    random.randint(0, 1)
    for _ in range(N)
]

print("\nBob's bases:")
print(bob_bases)

```

```

# =====
# 5. BOB MEASURES THE QUBITS
# =====

bob_bits = []

for i in range(N):

    qc = circuits[i]

    # Bob measures in X basis
    if bob_bases[i] == 1:
        qc.h(0)

    qc.measure(0, 0)

    result = simulator.run(
        transpile(qc, simulator),
        shots=1
    ).result()

    measured = list(
        result.get_counts().keys()
    )[0]

    bob_bits.append(
        int(measured)
    )

print("\nBob's measured bits:")
print(bob_bits)

# =====
# 6. BASIS SIFTING
#
# Alice and Bob publicly compare their bases.
#
# They KEEP only positions where:
#
# Alice basis == Bob basis
# =====

matching_positions = []

```

```

for i in range(N):

    if alice_bases[i] == bob_bases[i]:

        matching_positions.append(i)

print("\nMatching positions:")
print(matching_positions)

# =====
# 7. GENERATE RAW KEY
# =====

alice_key = [
    alice_bits[i]
    for i in matching_positions
]

bob_key = [
    bob_bits[i]
    for i in matching_positions
]

print("\nAlice raw key:")
print(alice_key)

print("\nBob raw key:")
print(bob_key)

# =====
# 8. ERROR DETECTION
#
# Compare a portion of the raw key.
# =====

if len(alice_key) > 0:

    test_size = max(
        1,
        len(alice_key) // 4
    )

```

```

test_positions = random.sample(
    range(len(alice_key)),
    test_size
)

errors = 0

for i in test_positions:

    if alice_key[i] != bob_key[i]:

        errors += 1

QBER = errors / test_size

print("\n=====")
print("QKD SECURITY CHECK")
print("=====")

print(
    "Test bits:",
    test_size
)

print(
    "Errors:",
    errors
)

print(
    "QBER:",
    round(QBER * 100, 2),
    "%"
)

# =====
# 9. SECURITY DECISION
# =====

if QBER == 0:

    print(
        "\nNo errors detected."
    )

```

```

        print(
            "Alice and Bob can continue"
            " with key generation."
        )

    else:

        print(
            "\nErrors detected!"
        )

        print(
            "Possible eavesdropping/noise."
        )

else:

    print(
        "\nNo matching bases."
    )

```

Qiskit version: 2.4.1

Alice's bits:

[0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1]

Alice's bases:

[1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1]

Bob's bases:

[0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Bob's measured bits:

[1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1]

Matching positions:

[2, 3, 6, 10, 11, 13, 19]

Alice raw key:

[1, 1, 0, 1, 0, 0, 1]

Bob raw key:

[1, 1, 0, 1, 0, 0, 1]

```

=====
QKD SECURITY CHECK
=====

```

Test bits: 1

Errors: 0
QBER: 0.0 %

No errors detected.
Alice and Bob can continue with key generation.

[]: